Sprint 4: VM Redundancy & Failover System

Status: Planning Phase — Ready for Implementation **Sprint Length:** 2 weeks **Start Date:** To be determined (team availability) **Epic:** Provider Reliability & VM Persistence **Estimated Total Effort:** ~46 hours (fits in 2 weeks with 3–5 developers working in parallel) **Design Reference:** 2026-03-09-vm-redundancy-design.md

---

🎯 Feature Overview

Problem: When a provider goes offline or a consumer's VM fails, all their data and installed packages are lost permanently. No failover, no recovery.

Solution: Implement a two-tier persistence system:

1. **User data** (`/home/consumer`) — synced to GCS every 5 minutes via background cron job
2. **System state** (OS + packages + configs) — committed as Docker images every 2 hours + on shutdown

When a consumer's provider fails, they can click "Migrate VM" to restore their entire environment on a healthy provider.

User Impact:

- Consumers see "Last backup: 3 min ago" timestamp
- Button: "Backup Now" (on-demand snapshot)
- Button: "Migrate VM" (when provider is unhealthy)
- On migration: Home directory + installed packages all restored automatically

---

📊 High-Level Architecture

```
Consumer Browser
    ├─ Requests volume size during VM creation
    ├─ Clicks "Backup Now" (optional, on-demand)
    └─ Clicks "Migrate VM" (when provider fails)
        ↓
Provider App (Local)
    ├─ Creates Docker volume (for /home/consumer)
    ├─ Injects GCP credentials into container at startup
    ├─ Monitors volume sync health (via Firestore)
    └─ Schedules container snapshots every 2 hours
        ↓
Docker Container (Running)
    ├─ Runs cron job every 5 min: gsutil rsync /home/consumer → GCS
    ├─ Runs FRP relay (background, root-only)
    └─ Consumer SSH access (unprivileged user, no access to GCP credentials)
        ↓
GCP Cloud Storage
    ├─ GCS Bucket: unicore-vm-volumes (user data backups)
    └─ Artifact Registry: unicore-vm-snapshots (container images)
```

```
          ↓
Firestore (State Machine)
    ├─ VirtualMachine (12 new fields for backup/migration status)
    ├─ VmMigrationRequest (tracks failover workflow)
    └─ Updated via provider app + heartbeat service
```

📋 Workstreams (Assignable Units)

This feature is broken into **7 workstreams**. Each can be assigned to a developer/pair and understood independently.

Total effort: ~46 hours distributed across team. **With parallel work (WS2, 3, 5 simultaneously), fits in 2-week sprint.**

#	Workstream	Type	Hours	Depends On	When
1	Data Models	Backend	2	Nothing	**Day 1** (critical path)
4	GCP Infrastructure	DevOps	2	Nothing	**Day 1** (parallel with #1)
2	Docker Volumes & Snapshots	Backend	8	#1	**Day 1–5** (after #1)
3	Consumer Volume Requests	Full-stack	6	#1, #2	**Day 1–5** (parallel with #2, #5)
5	Background Services	Backend	10	#1, #2, #4	**Day 1–5** (parallel with #2, #3)
6	Migration Orchestration	Backend	12	#1–#5	**Day 6–8** (after #1–5 complete)
7	Consumer Migration UI	Frontend	6	#1–#6	**Day 9–10** (after #6 complete)

🚀 Execution Sequence (2-Week Sprint)

Timeline: Complete in 2 weeks with parallel workstreams and a competent team.

Week 1: Foundation + Core Features (Parallel)

Start of Week (Day 1–2):

- **Workstream 1** (2 hrs) — Data models → *unblocks everything*
- **Workstream 4** (2 hrs) — GCP infrastructure → *independent parallel work*

After WS1 completes (Day 1–2 onwards), start in parallel:

- **Workstream 2** (8 hrs) — Docker volumes & snapshots
- **Workstream 3** (6 hrs) — Consumer volume requests
- **Workstream 5** (10 hrs) — Background services (monitoring + scheduling)

Expected completion: All 1–5 done by **end of Week 1** with parallel effort

Week 2: Integration + Polish

After WS1–5 complete (Day 6–7):

- **Workstream 6** (12 hrs) — Migration orchestration → *core business logic*

After WS6 complete (Day 8–9):

- **Workstream 7** (6 hrs) — Consumer migration UI → *final touches*

Expected completion: Everything done by **end of Week 2** with buffer time

Timeline Summary

Timeline	Workstreams	Status
Day 1–2	1, 4 (parallel)	Foundation
Day 1–5	2, 3, 5 (parallel)	Core features
Day 6–8	6 (sequential)	Integration
Day 9–10	7 (sequential)	Polish
Day 10+	Testing, integration, buffer	Final verification

Total effort: ~46 hours across team **Realistic with parallel work:** Fits in 2 weeks with 3–5 developers

---

🔄 Existing Patterns to Reuse

Your codebase already has well-established patterns. Each workstream reuses them:

Firestore Repositories

```
// Existing pattern in unicore.shared/Repositories/
public interface IFirestoreRepository<T>
{
    Task<T> GetByIdAsync(string id);
    Task<List<T>> WhereAsync(string fieldPath, object value);
    Task UpdateAsync(string id, T entity);
    // ... etc
}
```

Used by: Workstreams 1, 3, 6, 7 (all Firestore reads/writes)

Hosted Services (Background Work)

```
// Existing pattern in providerunicore/Services/ContainerMonitorService.cs
public class ContainerMonitorService : IHostedService
{
    public async Task StartAsync(CancellationToken ct) { }
```

```
    public async Task StopAsync(CancellationToken ct) { }
}
```

Used by: Workstreams 5, 6 (PeriodicTimer, background polling)

Firestore Real-Time Listeners

```
// Existing pattern in PauseResumeListenerService.cs
var listener = _vmRepository.CreateQuery()
    .WhereEqualTo("provider_id", providerId)
    .Listen(snapshot => { /* handle change */ });
```

Used by: Workstream 6 (migration request listener)

Docker Service Interface

```
// Existing pattern in providerunicore/Services/DockerService.cs
public interface IDockerService
{
    Task<string> StartContainerAsync(string name, string image, ...);
    Task StopContainerAsync(string containerId, ...);
    // ... to be extended in Workstream 2
}
```

Used by: Workstream 2 (new volume/snapshot methods)

Dashboard Request Processing

```
// Existing pattern in providerunicore/Components/Pages/Dashboard.razor
private async Task ProcessRequestAsync(VmRequest request)
{
    // Validate → Allocate → Docker → Firestore → Monitor
}
```

Used by: Workstream 3 (volume size integration), Workstream 6 (migration processing)

📄 Document Structure

Each workstream has a dedicated markdown file with:

1. **Overview** — What is this workstream?
2. **Context** — Why is it important?
3. **Architecture** — How does it fit?
4. **Integration Points** — Which files/services?

5. **What Needs to Be Done** — Specific tasks
6. **Code Examples** — Patterns from your codebase
7. **Acceptance Criteria** — How do we know it's done?
8. **Estimated Scope** — Lines of code, time estimate
9. **Blockers & Dependencies** — What must happen first?

🎓 For Your Team

When you meet with the team:

1. **Start with this file** — Gives everyone the 50,000-foot view
2. **Assign workstreams** — Based on interests/expertise
3. **Each person reads their workstream file** — Full context + acceptance criteria
4. **Discussion happens from an informed place** — Questions about "why" and "how," not "what"

🔗 Quick Links

- **Prerequisites & GCP Setup** — What must exist before we start code
- **Workstream 1: Data Models** — Firestore field additions
- **Workstream 2: Docker Volumes & Snapshots** — Docker service extensions
- **Workstream 3: Consumer Volume Requests** — Storage request feature
- **Workstream 4: GCP Infrastructure** — Buckets, accounts, keys
- **Workstream 5: Background Services** — Monitoring & scheduling
- **Workstream 6: Migration Orchestration** — Provider-side failover
- **Workstream 7: Consumer Migration UI** — UI for backup & failover

---

📞 Questions Before Starting?

Open questions to resolve with the team:

- ☐ Base Docker image pre-built or startup script? (Recommended: hybrid approach — pre-built + runtime script)
- ☐ What's the consumer's role in triggering migration? (Recommended: manual "Migrate VM" button, not automatic)
- ☐ Should backup snapshots be kept forever or have retention policy? (Recommended: keep latest 5, auto-delete older)
- ☐ How often should consistency score be updated? (Heartbeat service already does this every 10s)

Version: 1.0 **Last Updated:** 2026-03-14 **Owner:** UniCore Development Team